Design Rationale

Lab Script:

While creating the script for the lab, I first decided to use Ansible, Netmiko, and PyATS. I did this because I wanted to show how different modules in python work together to make a proper, working, piece of code. During the script creation, I decided to focus on netmiko due to issues with Ansible, and I realized that troubleshooting ansible was much more difficult than I would like to expose undergraduate students to.

The script was originally done in several different python files, tied to a main. I did this to portion off the different functions of the script, Backup, Change, Validate, Rollback. Backup would connect to the CSR1000v through netmiko, send the “show running-config” and capture the output. Using the python system time module, it would save the running config to a directory with a name formatted to include time and date the log was created. This was done in order to show how logs and backups are generally saved to more easily target a specific time, since an issue may be present in multiple iterations an update there needs to be a way to find the backup right before a faulty update.

Change was a system on input commands, asking the user “Do you want to change interface description” or similar, if there was a yes or y as the answer it would ask “What would you like to change it to?” This would take the user input, splice it into a command and send it to change the router. In this lab, I decided to only allow changing the description and motd banner to keep from causing harm to the environment. If the user responded with no to any of the prompts, it would skip that specific item. If all prompts were responded with no, the script would exit out automatically. Another design consideration I made while creating this and all other parts are errors, I implemented errors in the code so it would be easy to traceback where an issue originated from.

Validate was a simple script, it check the running config of the router and compares it to the current running configuration. Making sure to print which objects verified and which failed.

Rollback was another simple script, it would take all 3 objects from the backup and return them to the router. I decided to do this instead of just porting the entire backup running-config to avoid breaking the environment.

In the end, I realized that the current level of the lab was well above undergraduate and started condensing everything into a single script. In the lab script, the entire thing was created with immense commenting and explanations on the code with very frequent print statements meant to show students what is happening in real time. I removed the system time requirement from the

backup to reduce script complexity, and made sure it would be an option to copy and paste the code from the instructions. I still sectioned each function of the lab off into separate steps, with the code functionable without the proceeding sections, meaning students only ever have to add onto the existing file as they go. Example, section 1: Backup can run without the Change, Validation, and Rollback. Section 1&2 can run without Validation and Rollback.

As a final addition to the lab, I decided to add the over engineered script to the github and an optional section to the instructions. Letting students explore the more complicated code themselves.